OPERATING SYSTEMS LABORATORY

Bash Shell Scripting
Comparison Operators

Lecture Notes and Practical Reference

Learning objectives
Upon completion of this handout, students should be able to identify and use numeric, string, logical, and file-test operators in Bash conditional expressions.

1. Introduction

Comparison operators evaluate relationships between values or test conditions. Their results are commonly used with `if`, `elif`, and `while` statements to control the flow of a shell script. Bash provides different operators for numeric values, strings, logical expressions, and files.

2. Numeric comparison operators

Numeric comparison operators are used to compare integer values. In the traditional test syntax, the expression is enclosed in square brackets, with spaces required immediately inside the brackets.

Operator	Meaning	Example
-eq	Equal to	["$a" -eq "$b"]
-ne	Not equal to	["$a" -ne "$b"]
-gt	Greater than	["$a" -gt "$b"]
-lt	Less than	["$a" -lt "$b"]
-ge	Greater than or equal to	["$a" -ge "$b"]
-le	Less than or equal to	["$a" -le "$b"]

Example: Compare two integers

```
#!/bin/bash

read -p "Enter first number: " a
read -p "Enter second number: " b

if [ "$a" -gt "$b" ]; then
    echo "$a is greater than $b"
elif [ "$a" -lt "$b" ]; then
    echo "$a is less than $b"
else
    echo "Both numbers are equal"
fi
```

Note: These test operators are intended for integer comparisons. Validate user input before using it in arithmetic expressions.

3. String comparison operators

String operators compare text values or test whether a string is empty. Quoting variables is recommended to prevent errors when a value contains spaces or is empty.

Operator	Meaning	Example
=	Strings are equal	["$a" = "$b"]
!=	Strings are not equal	["$a" != "$b"]
-z	String has zero length	[-z "$a"]
-n	String has non-zero length	[-n "$a"]

Example: Username verification

```
#!/bin/bash

read -p "Enter username: " username

if [ "$username" = "admin" ]; then
    echo "Welcome, Admin."
else
    echo "Access denied."
fi
```

4. Logical operators

Logical operators combine conditions or reverse the result of a condition.

Operator	Meaning	Example
&&	Logical AND: both conditions must be true	["$a" -gt 10] && ["$b" -lt 20]
||	Logical OR: at least one condition must be true	["$a" -gt 10] \|\| ["$b" -lt 20]
!	Logical NOT: reverses a condition	! ["$a" -eq 10]

Example: Validate an age range

```
#!/bin/bash

read -p "Enter age: " age

if [ "$age" -ge 18 ] && [ "$age" -le 60 ]; then
    echo "Age is within the specified range."
else
    echo "Age is outside the specified range."
fi
```

5. File-test operators

File-test operators check whether a path exists and whether it refers to a particular file type or permission state.

Operator	Meaning
-e	The path exists (file, directory, or another filesystem object).
-f	The path exists and is a regular file.
-d	The path exists and is a directory.
-r	The path exists and is readable.
-w	The path exists and is writable.
-x	The path exists and is executable/searchable, as applicable.
-s	The path exists and has a size greater than zero.

Example: Check whether a regular file exists

```
#!/bin/bash

read -p "Enter file name: " filename

if [ -f "$filename" ]; then
    echo "The regular file exists."
else
    echo "The regular file does not exist."
fi
```

6. Arithmetic comparisons using (( ))

Bash supports arithmetic evaluation through double parentheses. Within `(( ))`, familiar mathematical comparison operators can be used directly.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
#!/bin/bash

a=20
b=10

if (( a > b )); then
    echo "A is greater than B."
else
    echo "A is not greater than B."
fi
```

Important: Inside `(( ))`, use arithmetic operators such as > and ==. Inside `[ ]`, use integer test operators such as `-gt` and `-eq`. For string comparisons, use = or `!=`.

7. Summary: Choosing the correct syntax

Purpose	Recommended syntax	Example
Compare integers	[ ] with -eq, -gt, -lt, etc.	["$a" -ge "$b"]
Compare strings	[ ] with = or !=	["$name" = "Ali"]
Test an empty string	[ ] with -z or -n	[-z "$name"]
Combine conditions	&& or ||	["$a" -gt 0] && ["$a" -lt 10]
Perform arithmetic comparison	(( )) with ==, >, <, etc.	((a >= b))
Test files and directories	[ ] with -e, -f, -d, etc.	[-d "$path"]

8. Student practice exercises

Write and execute Bash scripts to complete the following tasks. Use appropriate comparison operators and conditional statements.

1. Find the greater of two numbers.

2. Determine whether an integer is even or odd.

3. Check whether a student has passed or failed, using 50 as the passing mark.

4. Determine whether a number is positive, negative, or zero.

5. Compare an entered username with a predefined username.

6. Validate marks so that the value is between 0 and 100, inclusive.

7. Check whether a specified file exists and is a regular file.

8. Check whether a specified directory exists.

9. Determine whether a person is eligible to vote, using 18 as the minimum age.

10. Use a logical operator to check whether a number is within the range 1 to 100.

9. Execution procedure

Save the script with a `.sh` extension. For example, use `comparison.sh`. Then execute the following commands in a Linux terminal:

```
nano comparison.sh
chmod +x comparison.sh
./comparison.sh
```

Alternatively, a script can be run with `bash comparison.sh` without first changing its executable permission.

10. Good programming practices

• Include the interpreter declaration `#!/bin/bash` at the beginning of each script.
• Include spaces around the expression inside single square brackets, for example `[ "$a" -eq "$b" ]`.
• Quote string variables and file paths, for example `"$filename"`.
• Validate input before performing numeric comparisons.
• Use clear variable names and meaningful output messages.
• Close each `if` statement with `fi`.

Conclusion

Comparison operators are fundamental to decision-making in Bash Shell Scripting. A clear understanding of numeric, string, logical, and file-test operators enables students to develop reliable scripts for input validation, conditional processing, and basic file management.